Full Stack Development with AI

Lab 16.1 – Serving and Consuming ML Models Using API Endpoints

Lab Overview

Recall that model persistence and model serving are two important tasks that are required as part of the model productionisation process. In particular, serving machine learning (ML) models using API endpoints provides the greatest degree of flexibility and customisability. In this lab, you will learn how to serve Scikit-learn and Keras models using API endpoints and then consume the endpoints in software applications.

This lab will not cover the actual model training and persistence process. Instead, you will be provided with the persisted model files. The resources that you need for this lab can be found in "resources.zip".

Before proceeding, please ensure that you have the latest version of Python 3 and Node.js installed on your computer.

You will also require the knowledge and skills from previous lab exercises covering API endpoint development as well as backend and frontend software engineering.

Exercise 1 – Creating API Endpoints with Flask and Connexion

In this exercise, you will be creating two endpoints for serving two different ML models:

- An endpoint to serve a scikit-learn model for classifying Iris flower species that takes in four numerical independent variables as query string parameters.
- An endpoint to serve a Keras CNN model for classifying image shapes that takes in one uploaded file.

Follow the instructions to create a new Flask and Connexion application in Python:

1. Use the file explorer or terminal on your computer to create a new folder `solution`.
2. Start Visual Studio Code (VS Code) and open the newly created `solution` folder.
3. Create a new subfolder `app-api` folder.
4. Run the following command to create a new Python 3 virtual environment within the `app-api` folder.

```
python -m venv .venv
```

5. Follow the instructions to activate the newly created virtual environment:

For Windows command prompt:

```
.venv\Scripts\activate.bat
```

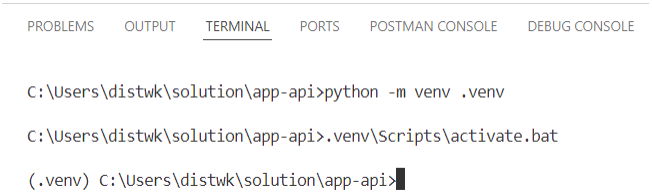
For Windows PowerShell:

```
.venv\Scripts\Activate.psl
```

For Linux and macOS:

```
source .venv/bin/activate
```

Your terminal should now resemble the following screenshot:



6. To exit from the virtual environment, use the `deactivate` command. But for now, please remain in the virtual environment.

7. Run the following `pip` commands to install some Python libraries within the newly created virtual environment:

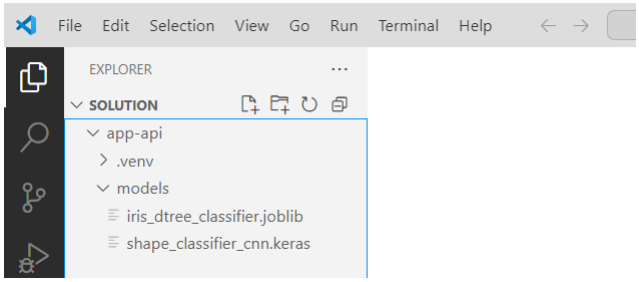
```
python -m pip install flask
python -m pip install connexion[flask,uvicorn,swagger-ui]
python -m pip install joblib
python -m pip install pandas
python -m pip install scikit-learn
python -m pip install tensorflow
python -m pip install pillow
```

8. Create a new subfolder `models` within the `app-api` subfolder.

9. Copy the following two model files from the resources folder that you have extracted into the `models` subfolder:

resources\models\iris_dtree_classifier.joblib
resources\models\shape_classifier_cnn.keras

10. The Explorer window of your VS Code should resemble the screenshot below:



We will now proceed to create a new endpoint to serve the scikit-learn model for classifying Iris flower species.

1. Create a `src` subfolder within the `app-api` subfolder.

2. Create a new file "app.py" in newly created `src` subfolder.

3. Paste the following code fragment representing the basic structure of a Flask and Connexion application into "app.py"

```
from connexion import FlaskApp

app = FlaskApp(__name__)
app.add_api("openapi.yaml")

if __name__ == "__main__":
    app.run(port=8000)
```

4. Create a new file "openapi.yaml" in the `src` subfolder.

5. Paste the following configuration into "openapi.yaml"

```
openapi: "3.0.0"

info:
  description: API Endpoints for serving Machine Learning Models
  version: 1.0.0
  title: API Endpoints for serving Machine Learning Models

servers:
  - url: http://localhost:8000/api

paths:
```

6. Create a new file "models.py" in the `src` subfolder. This is the Python source file containing the endpoint handler functions.

7. Paste the following code fragment into “product.py”

```
import json
from flask import Response
from flask import request
```

8. In “models.py”, define a Python handler function `irisClassifier` that performs the following four tasks:

    a. Retrieves the data for a new Iris flower observation containing the four independent variables of sepal length, sepal width, petal length and petal width from the body of the request in JSON object format.
    b. Load the model file “models\iris_dtree_classifier.joblib” and pass the independent variable values to the model to obtain the prediction result.
    c. Return the prediction result together with the HTTP 200 status code.
    d. If an error occurs, return an appropriate error message together with the HTTP 500 (Internal Server Error).

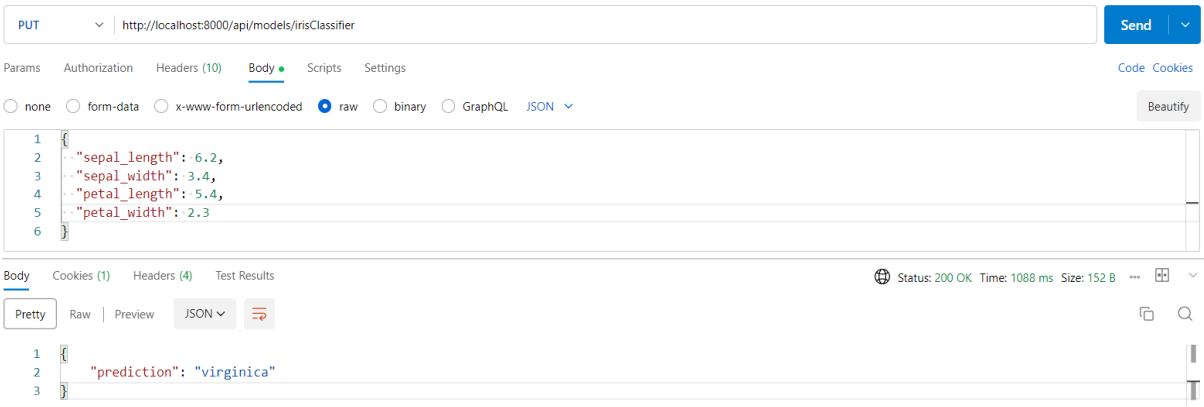
9. In “openapi.yaml”, define a `PUT` endpoint to the path `/models/irisClassifier` that maps to the Python handler function `irisClassifier` defined in step (8).

10. Start a new terminal in VS Code and change into the `app-api` subfolder and then activate your Python virtual environment.

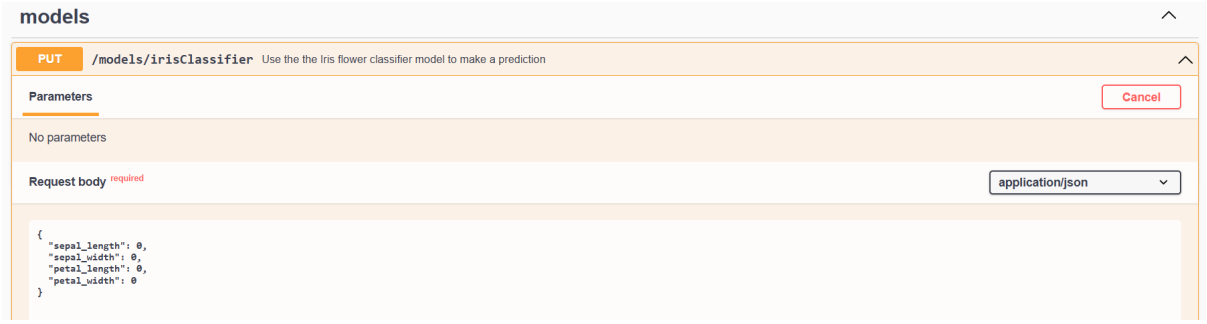
11. Run the following command to run the web application:

```
cd src
python app.py
```

12. Use Postman to test the endpoint by inputting a sample Iris flower observation in the request body using the JSON format:



13. Navigate to http://localhost:8000/api/ui to view the Swagger UI dashboard and test the endpoint from the dashboard too.



Let's create another new endpoint to serve the Keras model for classifying image shapes.

1. Create a `temp` subfolder within the `app-api` subfolder.

2. In "models.py", define a Python function `preprocess_image` to preprocess an image file uploaded by user before invoking the Keras model for the shape classification. The code for `preprocess_image` is provided below.

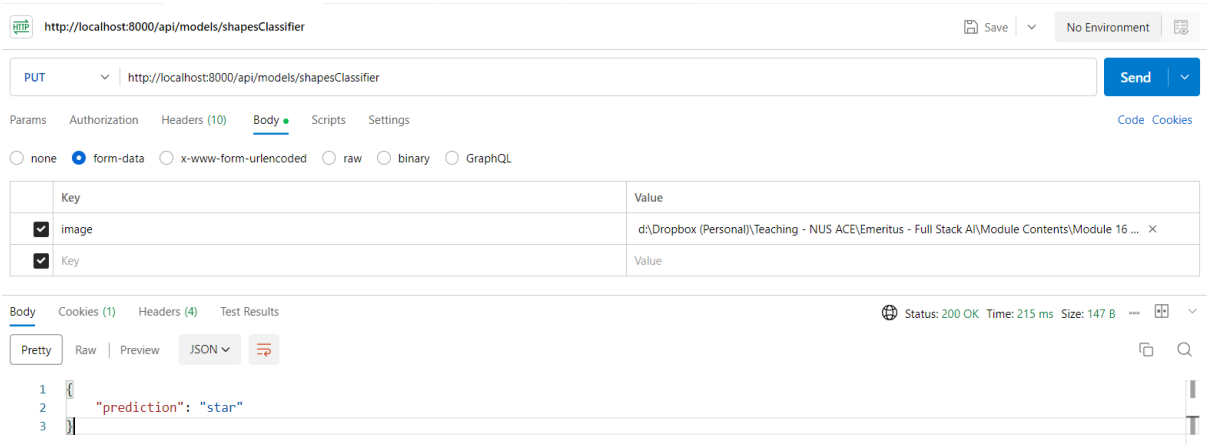
```
def preprocess_image(image_path, img_size=IMG_SIZE):

    img = load_img(image_path, target_size=img_size)
    img_array = img_to_array(img)
    img_array = img_array / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    return img_array
```

3. In "models.py", define a Python handler function `shapesClassifier` that performs the following five tasks:

    a. Retrieves the uploaded image file from the request and save it temporarily to the `temp` subfolder that you have created earlier.
    b. Preprocess the uploaded image file using the Python function `preprocess_image` that you have created earlier.
    c. Load the model file "models\shape_classifier_cnn.keras" and pass the preprocessed image array to the model to obtain the prediction result.
    d. Return the prediction result together with the HTTP 200 status code.
    e. If an error occurs, return an appropriate error message together with the HTTP 500 (Internal Server Error).

4. In "openapi.yaml", define a `PUT` endpoint to the path `/models/shapesClassifier` that maps to the Python handler function `shapesClassifier` defined in step (3).

5. Run the web application and test the endpoint with Postman.



6. When you done, you can stop the web application running the API endpoints by pressing Ctrl+C and exiting from the virtual environment.

   Alternatively, you can leave the web application running as we will be building some frontend web applications with Python and JavaScript in the subsequent exercises.

Exercise 2 – Creating Server-side Rendering Web Application with Python and Flask to Consume the API Endpoints

In this exercise, you will create a Python Flask web application that consumes the API endpoints created in Exercise 1. You will also be using the Jinja2 templating engine to create the views of the web application. In gist, at the end of this exercise, you will have created a complete full-stack AI solution involving ML models as well as the backend and frontend of software applications. You will find these skills to be especially useful for your capstone project.

Follow the instructions to create a new Flask application in Python:

1. Create an `app-web` subfolder within the `solution` folder.

2. Create a new Python 3 virtual environment within the `app-web` folder.

   If you want to, you can launch a new terminal window at this juncture and activate the virtual environment. Later, we will need to run the Python web application within the virtual environment.

3. Run the following `pip` commands to install some Python libraries within the newly created virtual environment:

```
python -m pip install flask
python -m pip install requests
```

4. Create the following subfolders within the `app-web` subfolder:

   - `src` subfolder
   - `templates` subfolder
   - `static\images` subfolder (the `images` subfolder should reside in the `static` subfolder)

5. Create a new file "app.py" in newly created `src` subfolder.

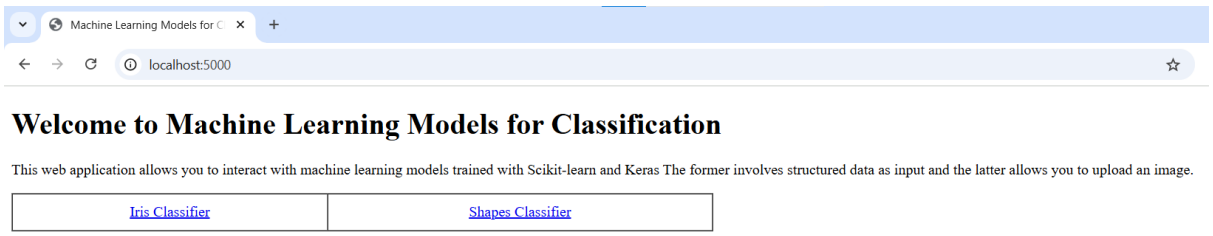
7. Paste the following code fragment representing the basic structure of a Flask application into "app.py"

```
from flask import Flask
from flask import render_template
from flask import request
import requests
import os

app = Flask(__name__, template_folder='../templates',
static_folder='../static')

UPLOAD_FOLDER = '../static/images'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
```

6. Add a `GET` endpoint to the root path `/` that renders the Jinja2 template "index.html" as response.

7. Create a new file "index.html" in the `templates` folder.

8. Input the HTML code for "index.html" to create a web page resembling the figure below:



9. Add a `GET` endpoint and another `POST` endpoint to the root path `/irisClassifier` that perform the following tasks:

    a. `GET` endpoint – Renders the Jinja2 template "irisClassifier.html" as response.

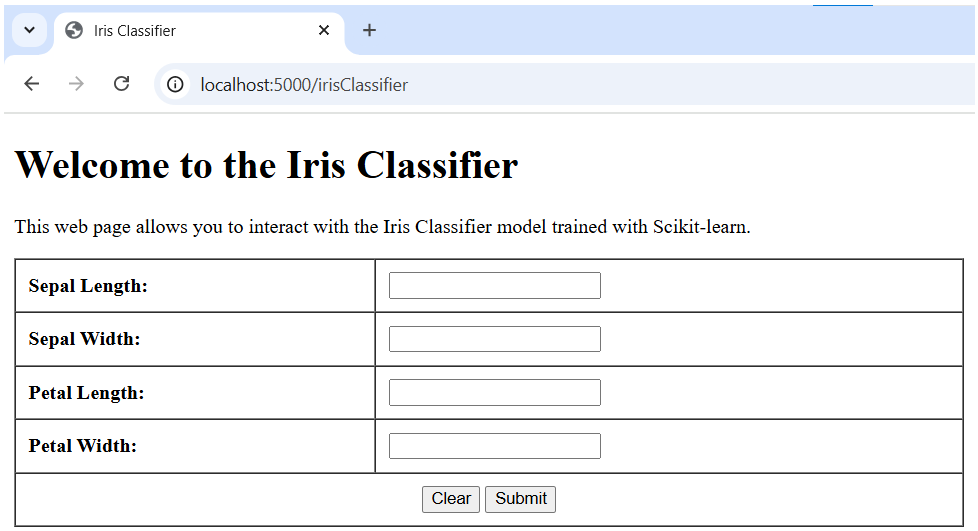
    b. `POST` endpoint – Retrieves the form input data for the four independent variables of sepal length, sepal width, petal length and petal width.

    Then invoke the `PUT` endpoint to the path `http://localhost:8000/api/models/irisClassifier` to obtain the prediction result.

    Renders the Jinja2 template "irisClassifier_prediction.html" as response.

10. Create a new file "irisClassifier.html" in the `templates` folder.

11. Input the HTML code for "irisClassifier.html" to create a web page resembling the figure below:



12. Create a new file "irisClassifier_prediction.html" in the `templates` folder.

13. Input the HTML code for "irisClassifier_prediction.html" to create a web page resembling the figure below:

Iris Classifier Prediction Result

Sepal Length:	6.2
Sepal Width:	3.4
Petal Length:	5.4
Petal Width:	2.3
Prediction:	virginica

14. Add a `GET` endpoint and another `POST` endpoint to the root path `/shapesClassifier` that perform the following tasks:

    a. `GET` endpoint – Renders the Jinja2 template "shapesClassifier.html" as response.

    b. `POST` endpoint – Retrieves the form input data for the uploaded image file.

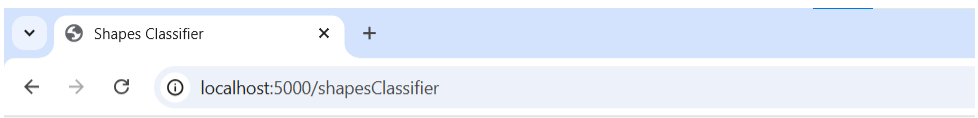
    Save the image file so that it can be displayed later together with the prediction result.

    Then invoke the `PUT` endpoint to the path `http://localhost:8000/api/models/shapesClassifier` to obtain the prediction result.

    Renders the Jinja2 template "shapesClassifier_prediction.html" as response.

15. Create a new file "shapesClassifier.html" in the `templates` folder.

16. Input the HTML code for "shapesClassifier.html" to create a web page resembling the figure below:



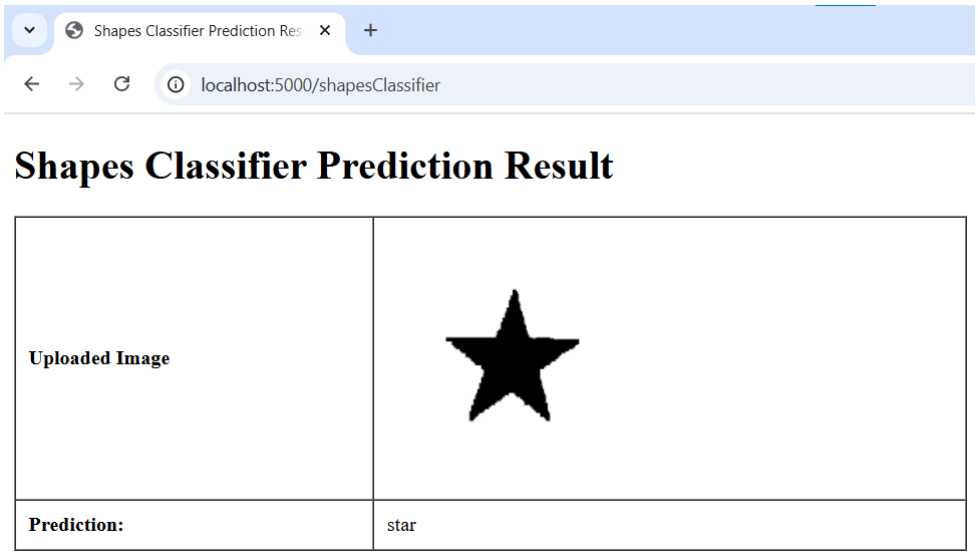
Welcome to the Shapes Classifier

This web page allows you to interact with the Shapes Classifier model trained with Keras.

Upload Image:	Choose File No file chosen
Clear Submit	

17. Create a new file "shapesClassifier_prediction.html" in the `templates` folder.

18. Input the HTML code for "shapesClassifier_prediction.html" to create a web page resembling the figure below:



19. Run the following command to run the web application:

```
python -m flask --app app run
```

20. Test the web application and ensure that both models are working well.

    There are some sample images in "resources\images" that you can use to test the shapes classifier.

-- End of Lab --